Week 06 Rust Alpha Beta Pruning and Transposition Tables

Project Research Design and Implementation of a Gomoku Game Engine with Artificial Intelligence
Academic module Basic Project
Report language English
Evidence review date 17 October 2026

1 Research purpose and evidence boundary

This week asks whether Alpha Beta can reduce work while preserving the finite depth Minimax value established in Week 05. The comparison must keep the game tree fixed. Changing candidate radius, move cap, terminal rules, evaluation weights, or depth at the same time would confound the result. Transposition tables introduce a second question: which cached results can be reused without changing the meaning of a search?

Evidence labels distinguish **[A]** inspected source or recorded execution, **[B]** primary literature, **[C]** deduction or worked example, and **[D]** proposed work. This report concerns the native reference crate and a local research harness. Its presence does not establish that the optimized search is connected to the React game or its existing Wasm artifact.

Earlier synthetic tree values and benchmark tables are replaced. In particular, a MAX root with MIN children containing leaves `[3,5]` and `[6,9]` has value six, not three. A valid pruning demonstration must show its leaf order and derive its counts rather than infer them from a diagram caption.

2 Questions and primary literature

The research questions are:

1. Under identical finite tree semantics, do plain Minimax and Alpha Beta return equal root scores?

2. How do ordering, equality cutoffs, and root tie retention affect work and selected moves?

3. When is a table entry exact, an upper bound, or a lower bound, and what context makes reuse valid?

[B] Knuth and Moore give a correctness argument for Alpha Beta and analyze its search cost under ordering assumptions. Their result concerns preservation of a specified tree value, not the correctness of a heuristic evaluation or an omitted move policy. Ideal ordering can greatly reduce leaf examination; the ideal case is not an unconditional runtime guarantee. Knuth and Moore 1975 An Analysis of Alpha Beta Pruning.

3 Source responsibilities and baseline controls

[A] `caro-core-reference/src/search/alpha_beta.rs` contains `search_alpha_beta`, recursive `value`, and `terminal_score`. It shares `SearchConfig`, `generate_moves`, `GameState::play`, `GameState::undo`, and `evaluate` with Minimax. It checks terminal state before depth zero, carries a fixed root perspective, and restores a played move before updating bounds or breaking.

Source relative to the reference crate	Responsibility and observed behaviour
`src/search/alpha_beta.rs`	Basic bounds cutoffs and deterministic root selection
`src/search/ordering.rs order_moves`	Temporary play evaluation undo and deterministic sorting

Source relative to the reference crate	Responsibility and observed behaviour
`src/search/optimized.rs`	Table probes stores incremental keys and node budget aborts
`src/search/transposition.rs`	Entry depth score bound and preferred move
`src/zobrist.rs` `hash` and `after_play`	Board and side keys with incremental updates
`tests/search_equivalence.rs`	Existing plain Minimax and Alpha Beta parity fixture

[D] A clean baseline disables ordering and table reuse and selects the same move mode in both algorithms. Subsequent comparisons add ordering alone, then a cold table, then a compatible warm table. A cold table has no stored entries; a warm table contains entries from a declared preceding search. These are separate treatments.

4 Alpha Beta bounds and correctness

[C] Write $V(s)$ for the fixed depth Minimax value with its depth, root, generator, and evaluator context understood. Alpha and beta are relevance thresholds inherited from ancestors. They do not assert that the unknown node value necessarily lies between them.

At a MAX node, the best examined child is a lower bound on the full node value. At a MIN node, the best examined child is an upper bound. Suppose a MIN node receives alpha equal to six because an ancestor MAX already has an alternative of at least six. If one child of this MIN node returns at most five, its final minimum cannot exceed five, regardless of unexamined children. The ancestor need not know whether the exact minimum is five, three, or negative one.

The dual argument applies at MAX: once one child reaches beta, the parent MIN already has an alternative no larger than beta. Additional siblings cannot make the MAX branch preferable to that alternative. Both cases permit the cutoff

$$\alpha \geq \beta .$$

Equality can prune because a tied branch cannot strictly improve the established ancestor choice. It is relevant to tie conventions, however: pruning does not enumerate every optimal move.

[C] Correctness follows by induction on remaining depth. Leaves return the same score as Minimax. Internal children either return their exact relevant value or a bound sufficient to refute their branch. A cutoff discards only children whose possible results cannot change the ancestor decision. A full window root therefore obtains the same finite tree score. This argument assumes valid state restoration and compatible leaf values; it does not validate the heuristic against the real game outcome.

For an initial finite window (α_0, β_0), a fail soft return r is interpreted conservatively as

$$\begin{cases} V(s) \leq r, & r \leq \alpha_0, \\ V(s) = r, & \alpha_0 < r < \beta_0, \\ V(s) \geq r, & r \geq \beta_0. \end{cases}$$

A cutoff result is consequently not always the exact node value. The implementation uses integer extrema for the full root window; actual scores remain inside those extrema.

5 Source faithful traversal summary

[A] Plain Alpha Beta updates alpha from its best MAX score and beta from its best MIN score. It returns the best score found rather than replacing every cutoff result with a window endpoint. The following pseudocode summarizes the shared recursive structure; generated moves are legal, and optional ordering occurs before traversal.

```
AB state h root ply alpha beta
  count node; if terminal return terminal utility
  if h equals zero return heuristic
  moves = generate moves; if empty return heuristic
  optionally order moves; save whether parent is MAX
  best = minus infinity for MAX otherwise plus infinity
  for move in moves
    token = play move; child = AB state h minus one root ply plus one alpha beta
    undo token; best = max or min of best and child
    update alpha at MAX or beta at MIN
    if alpha at least beta record cutoff and break
  return best
```

[A] Root replacement uses strict improvement, as in Minimax. With the same root ordering, the first optimal move remains selected. Score equality alone does not imply identical moves when ordering differs. `order_moves` sorts descending root perspective score at every node and uses row major indices for score ties. Descending root scores may be poor ordering at a MIN node, but ordering without omission does not alter its mathematical value.

6 Concrete synthetic trees

[C] The following depth two trees have a MAX root and ordered MIN children. Each bracket is one child list; traversal is left to right. Counts include the root, internal nodes, and visited leaves. These are analytical examples, not trees executed by the native harness.

Tree	Ordered MIN leaves	Value	MM nodes	AB nodes
A	`[3,5] [6,9]`	6	7	7
B	`[9,6] [5,3]`	6	7	6
C	`[1,2,3] [4,5,6] [7,8,9]`	7	13	13
D	`[7,8,9] [4,5,6] [1,2,3]`	7	13	9
E	`[3,5] [3,9]`	3	7	6

For A, the first child gives three. The second begins at six, still greater than alpha three, so nine must be inspected before its minimum six is known. No pruning occurs. For B, the first minimum is six; the second child sees five and immediately refutes improvement, skipping three.

For D, the first minimum is seven. The next two MIN children are refuted by four and one respectively. Five leaves and four internal nodes are visited, giving nine nodes. C has the same root value but an order that causes no reduction. In E, equality at three is enough to skip nine, while the first root move remains the selected tie.

These calculations also distinguish a cutoff event from work saved. D has two cutoffs and four skipped sibling leaves. In a deeper tree, one skipped sibling could contain many descendants; the two measures are not interchangeable.

7 Table entries and bound semantics

[A] `TTEntry` contains remaining depth, score, `Bound::{Exact,Lower,Upper}`, and an optional best move. `optimized.rs::value` saves the original window, probes the table after terminal and horizon handling, and accepts a score entry when stored depth is at least requested depth. An exact entry returns immediately. A lower bound raises alpha; an upper bound lowers beta; a closed window permits a return.

For storage, the code classifies the returned score against the original window:

$$Bound(r)=\begin{cases} Upper, & r\le\alpha_0, \\ Lower, & r\ge\beta_0, \\ Exact, & \alpha_0<r<\beta_0. \end{cases}$$

[C] For example, a MIN search with alpha six that cuts off after a child scoring five has established an upper bound of five. Storing that five as exact could later conceal the actual minimum three in a wider search. Bound tags preserve the distinction. The preferred move can guide ordering even when the score depth is insufficient, which explains why move reuse and score reuse need different acceptance rules.

[A] The table is a `HashMap<u64,TTEntry>`. Replacement keeps an existing strictly deeper entry but otherwise accepts the new one. Reserved capacity is not a hard memory limit; the table can grow. This report makes no bounded memory claim.

8 Context required for safe reuse

[A] The Zobrist key XORs piece keys and a side key. `after_play` toggles the side key only if the actual state transition changes side, accounting for terminal moves that keep the mover. Its existing unit test checks one ordinary move against full recomputation.

[C] A matching 64 bit key is not a proof of equal state: collisions remain possible, and the entry stores no full board verification. More directly, the key omits rule profile, root evaluation perspective, weights, candidate policy, and mate distance context. Reusing an external table after any of these changes can reinterpret an incompatible score. A report must therefore declare the table lifetime, rather than assume board equality alone is sufficient.

Stored depth at least requested depth is a conventional engine policy, but deeper heuristic search need not equal a shallower fixed horizon value. Strict same depth equivalence experiments should use a fresh table or entries known to represent the same horizon semantics. Within one ordinary placement search, transposed equal boards have equal occupied counts and therefore equal root ply. Moving the root for another search changes that guarantee: a stored $M-\ell$ mate score needs normalization or a compatible context before reuse.

[D] A robust integration should scope or clear tables by search context, establish mate normalization, test collision handling policy, and document any deliberate deeper result substitution. These are concrete obligations derived from the inspected key and entry formats, not claims that a visible game failure has been reproduced.

9 Experiment method and result interpretation

Run `python3 research/week_05_09/run.py` from the workspace root. Its provenance records locked commands, source hashes, Rust 1.98.1, and native release execution on Linux x86 64 with an Intel Xeon W 11855M. Timing excludes cloning and table or key initialization; controlled fixtures use three repetitions. Raw rows and summaries retain individual configurations. The plain comparison holds depth, weights, rules, generation, and ordering fixed; optimized comparisons additionally declare cold or warm table state.

The node reduction measure is

$$R_N = 1 - \frac{N_{AB}}{N_{MM}}.$$

[C] For synthetic tree D, $R_N = 1 - 9/13 = 30.77\,\%$. This is a node reduction, not a measured speedup. Elapsed speedup requires T_{MM}/T_{AB} under comparable measurement conditions.

[A] `SearchStats::skipped_siblings` counts remaining sibling moves at cutoff points, not all omitted descendants. `tt_hits` counts entries passing the depth check, and horizon leaves are evaluated before probing. Root preferred move access is not counted in the same lookup counter as recursive probes. Instrumentation therefore needs its source definition alongside any result.

10 Recorded parity and cache scope results

[A] Natural ordering with table reuse disabled produced these paired results. Both algorithms returned equal scores and identical moves within each row.

Fixture and mode	Depth	MM nodes	AB nodes	Reduction
Clustered all legal	2	48842	10939	77.60 percent
Win all legal	2	47526	11922	74.91 percent
Clustered radius 1 cap 12	3	1885	402	78.67 percent

[A] Verification records 112 plain score and move comparisons, 112 ordering score comparisons, and 208 cold table score comparisons across four controlled and sixteen seeded positions, including repetitions. All passed. Ordering comparisons assert scores, not identical tied moves. The shared harness checked root restoration 551 times; nine reference crate tests passed.

[A] At clustered radius one cap eight depth four, plain Alpha Beta visited 1094 nodes and cold table search 1088, with four hits from 255 recursive lookups. Median local times were 774 and 812 microseconds respectively, so this sample does not demonstrate a table speedup. Table semantics evidence also records exact, lower, and upper entries.

[A] The cache scope probe found same root warm reuse preserved score zero with nine nodes and eight hits. After playing the selected move and changing the root player, a fresh depth three search scored zero while reuse scored 19800. This is a reproduced native context mismatch. The key omissions in Section 8 explain why arbitrary reuse is unsupported; the probe does not isolate every omitted context component. Use a fresh table per root context until that contract is repaired.

11 Limitations and conclusions

[A] The existing parity test covers a tactical fixture at depth two with all legal moves and ordering disabled. It checks both score and move; the immediate win test checks row seven column two. That coverage does not justify a statement that all positions or all table configurations are equivalent.

[C] Alpha Beta is value preserving relative to its declared tree, while candidate omission and horizon evaluation change the question being answered. Ideal ordering can reduce a uniform tree to a leaf count of order $b^{h/2}$; poor ordering can still require order b^h. Actual evaluator work, ordering evaluations, table cost, and build mode determine elapsed performance.

[D] The defensible next step is to retain plain Minimax as a shallow oracle, compare exact same tree configurations, and add explicit incompatible context and mate score reuse tests before table integration. Native correctness evidence remains separate from Wasm generation, browser Worker behaviour, cancellation, and playing strength.